

Database Migrations — Core Concept

In a large project, database changes should be version-controlled using migration files. Developers do not manually maintain one shared schema file or execute every SQL script one by one.

1. Core Concept

Migration = a version-controlled database change.

Migration file = the script that makes that change.

Migration history = the database's record of which migrations have already been applied.

npm run migrate = checks the history and applies only pending migrations.

npm run dev = starts the application after the database is up to date.

```
Developer makes DB change
↓
Create a new migration file
↓
Commit migration file to Git
↓
Other developers pull the changes
↓
Run: npm run migrate
↓
Migration tool checks migration history
↓
Runs only new migrations
↓
Records them as completed
↓
npm run dev
```

2. Migration Files

Database changes are kept in a migrations folder. Each file represents a database change.

```
database/
  ■■■ migrations/
    ■■■ 001_create_users.sql
    ■■■ 002_create_products.sql
    ■■■ 003_create_orders.sql
    ■■■ 004_add_email_to_users.sql
```

3. Creating a New Table

```
-- 001_create_users.sql

CREATE TABLE users (
  id INT PRIMARY KEY,
  name VARCHAR(100)
);
```

4. Adding a New Column

Do not modify the old migration file. Create a new migration for the new change.

```
004_add_email_to_users.sql

ALTER TABLE users
ADD email VARCHAR(255);
```

5. Commit the Migration

```
git add database/migrations/004_add_email_to_users.sql
git commit -m "Add email to users"
git push
```

6. Migration History

Each developer normally has their own local database. The migration tool maintains a special history table in that database, commonly called something like `migration_history`.

```
migration_history

001_create_users
002_create_products
003_create_orders
```

7. What Happens After Git Pull?

Suppose the project now contains migration 004, but your local database has only recorded 001–003.

```
Migration files:
001 ✓
002 ✓
003 ✓
004 ✗

Local migration history:
001 ✓
002 ✓
003 ✓
```

8. Run the Migration

```
npm run migrate
```

The migration tool compares the migration files with the migration history and runs only the missing migration.

```
004_add_email_to_users.sql
↓
Execute SQL
↓
Record 004 in migration_history
```

9. Then Start the Application

```
git pull

npm run migrate

npm run dev
```

You do not manually execute every SQL file. The migration tool determines which migrations are still pending.

10. What Happens on the Next Run?

```
npm run migrate

001 ✓
002 ✓
003 ✓
004 ✓

→ Nothing runs because all migrations are already recorded.
```

11. Most Important Team Rule

- Do not modify old migration files.
- Create a new migration for every new database change.

001_create_users.sql	← keep unchanged
002_create_products.sql	← keep unchanged
003_create_orders.sql	← keep unchanged
004_add_email_to_users.sql	← new change
005_add_phone_to_users.sql	← another new change

Quick Summary

Migration files are the database's version history. Git shares those changes with the team. Each local database keeps track of which migrations it has already applied. **npm run migrate** applies only pending changes, and then **npm run dev** starts the application.